Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

Allocating inference compute under a fixed budget

Zhenbang Lyu

Heidelberg University
Institute of Computer Science

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar
2024, arXiv:2408.03314v1

Can Thinking Longer Improve an Answer?

A bat and a ball cost \$1.10 in total. The bat costs \$1.00 more than the ball. How much does the ball cost?

- **A quick guess** The ball costs \$0.10. But then the bat costs \$1.10, giving a total of \$1.20.
- **After checking** Let the ball cost x. Then $x + (x + 1) = 1.10$, so the ball costs \$0.05.

Extra thought helps us check and revise an initial answer.

Can a trained language model benefit in a similar way?

How Can a Model Use More Computation?

For a model, extra effort means more computation while answering, after training: **test-time compute**.

Suppose the budget allows 16 candidate answers to a math problem.

- Generate 16 independent attempts and select an answer.
- Generate a chain of 16 attempts, each conditioned on earlier answers.
- Spend the budget searching over partial solutions with a verifier.

Which allocation gives the highest chance of a correct final answer?

Each option needs a way to generate candidates and a way to choose among them.

The Proposer and Verifier Framework

The paper describes these choices through two components.

Proposer The language model generates candidate solutions from a probability distribution.

Verifier A learned model scores candidates or partial solutions.

- **Revisions** change the proposal distribution by including earlier answers in the next input.
- **Verifier-guided search** uses scores to select candidates or decide which prefixes to expand.

We will examine both routes, then ask whether a small model using them can outperform a larger model at equal total FLOPs.

Experimental Setup

To test these ways of using extra computation, the authors study mathematical reasoning.

- **Task** MATH, high-school competition mathematics with a range of difficulty levels.
- **Data split** 12,000 training questions and 500 test questions.
- **Base model** PaLM 2-S* (Codey), with task-specific fine-tuning for verification and revision.

The model already solves some problems. We now need to decide how much exploration or refinement each question needs.

Choosing a Strategy by Difficulty and Budget

A promising first answer may benefit from revision. A poor starting point may need a fresh attempt.

The authors use **difficulty relative to the base model** to choose settings for each budget.

1. Estimate difficulty from 2,048 independent samples per question.
2. Sort questions into five equal-frequency bins, from easy to hard.
3. For each bin and budget, select a setting on one fold and evaluate on the other. Swap folds and average.

Oracle bins Rank by actual correctness rate.
Predicted bins Rank by the verifier's mean final-answer score.

Reported budgets exclude difficulty-estimation cost.

Training a Process Reward Model

To choose which partial solutions to develop, search needs a score for each prefix.

A **process reward model (PRM)** estimates the chance of eventual success under the base model's sampling policy.

- Keep a solution prefix through the current step.
- Sample multiple continuations to completion and check final correctness.
- Use the success rate as a training target for the PRM.

Illustrative training target: 7 correct completions out of 10 gives 0.7.

- Their PRM outperformed an outcome reward model (ORM) baseline, so subsequent search experiments use the PRM.

PRM Scores and Final Answer Selection

Training gives us scores for partial solutions. We also need a rule for choosing the final answer when candidates are complete.

Within a solution Use the PRM score at its last step as the full-solution score.

Across solutions Weighted best-of-N sums scores for candidates with the same final answer.

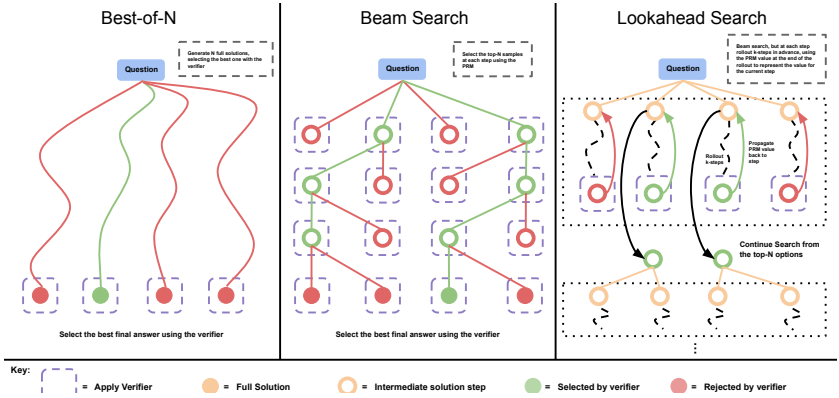
Example candidate	Final answer	Full-solution score
1	$x = 2$	0.60
2	$x = 2$	0.55
3	$x = 3$	0.90

$$x = 2\text{: } 0.60 + 0.55 = 1.15 \qquad x = 3\text{: } 0.90$$

Weighted selection returns $x = 2$. Majority voting counts occurrences without these weights.

Search Methods with a PRM

The same scorer can evaluate complete answers or guide generation before a solution is finished.

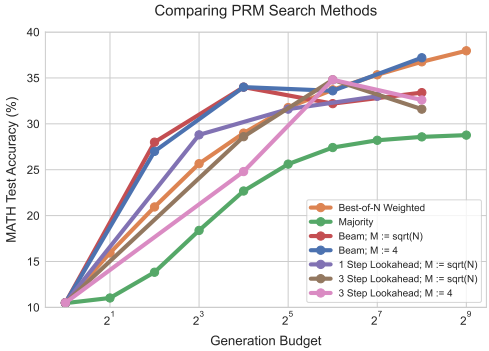


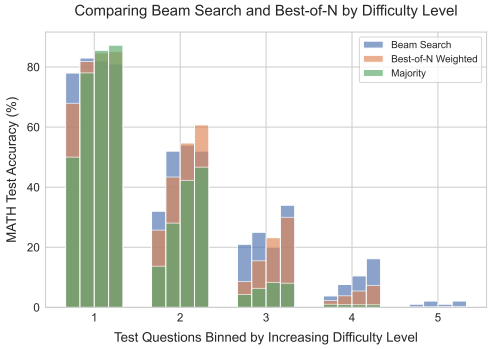
Best-of-N Complete independent candidates, then score.

Beam search Score prefixes, retain top candidates, expand again.

Lookahead Continue k steps temporarily before scoring a prefix. Its extra rollout also consumes budget.

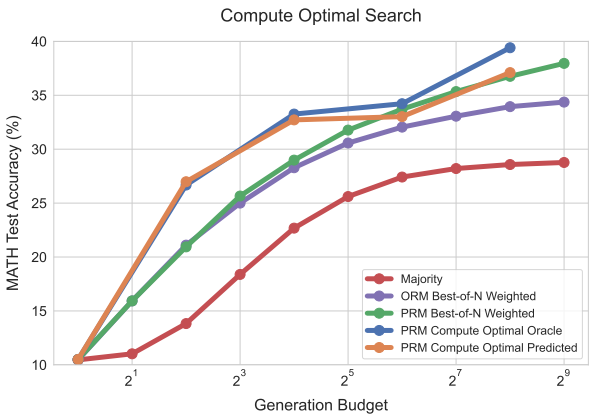
Search Results Depend on Budget and Difficulty





- Low budgets favor beam search. At higher budgets, its advantage shrinks.
- Breaking down by difficulty explains the difference: easy bins 1–2 can deteriorate as search exploits PRM errors, while beam search helps in bins 3–4.
- The hardest bin improves little. Lookahead usually trails at matched cost.

Difficulty-Based Allocation Improves PRM Search



Since different settings win in different bins, select the best search setting for each bin and budget.

At low budgets, 16 generations nearly match best-of-N with 64.

With predicted bins, some gains diminish at higher budgets.

Can we also improve what the model proposes by letting it revise earlier answers?

Training and Running a Revision Model

The second route lets the generator use earlier answers to produce a better next attempt.

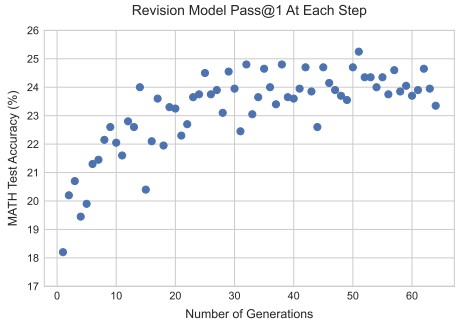
Training data

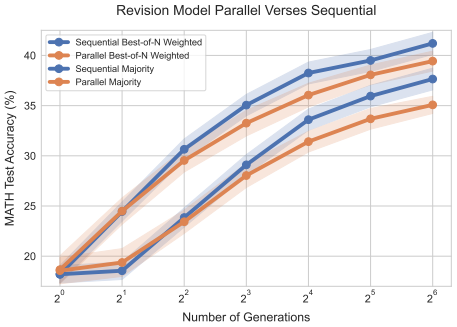
- Generate 64 independent answers per training question at higher temperature.
- Pair a correct target with 0–4 incorrect answers, prioritizing related text using character edit distance.
- Fine-tune the model to produce the correct target given the question and earlier answers.

At inference

- Generate a chain, keeping the most recent four answers in context.
- Revisions can undo correct answers. Select among candidates using a verifier or majority vote.

Revision Quality and Final Answer Accuracy

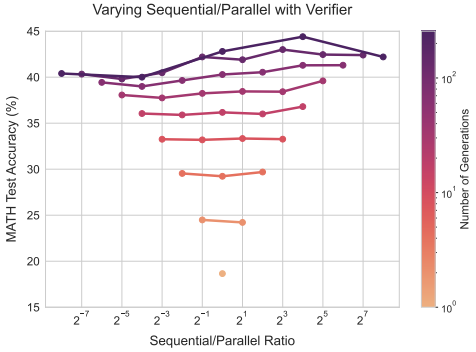


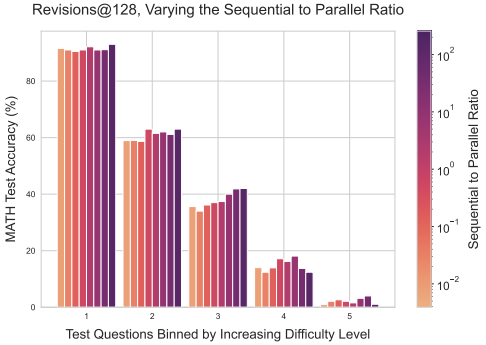


- **Left** Single-answer accuracy improves overall with revision, then fluctuates around a plateau.
- **Right** At the same candidate count, sequential generation generally beats parallel generation with either selection method.

Could several independent revision chains work better than one long chain?

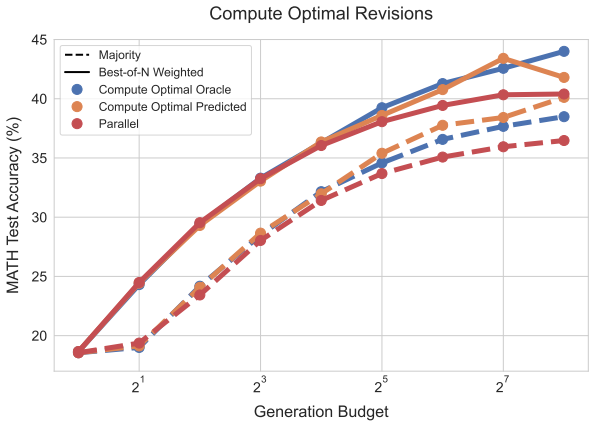
Combining Fresh Starts with Revisions





- With 16 candidates: 16 chains of length 1, four of length 4, or one of length 16.
- **Left** Each line fixes the budget. Moving right puts more compute into each chain.
- **Right** At 128 candidates, easy bins favor long chains. Harder bins can favor a mixture.

Compute-Optimal Revisions Improve Scaling



Apply the same difficulty-based rule to the sequential/parallel ratio.

Parallel sampling begins to plateau at high budgets.

Adaptive revisions can beat parallel best-of-N with 64 rather than 256 candidates.

How far can these gains close the gap to a larger model?

Pretraining and Inference at Equal Total FLOPs

Extra inference improves the small model. How does it compare with training a larger model?

Hold training tokens fixed and increase parameters by a factor M.

$$X \approx 6PD_{\text{pretrain}}, \qquad Y \approx 2PD_{\text{inference}}$$

P is the parameter count. $D_{\text{inference}}$ is the total inference token workload.

$$\underbrace{X + aY}_{\text{small model, extra inference}} = \underbrace{M(X + Y)}_{\text{large model, greedy decoding}}$$

The larger model costs more both to train and to run. The small model can spend its training savings on inference.

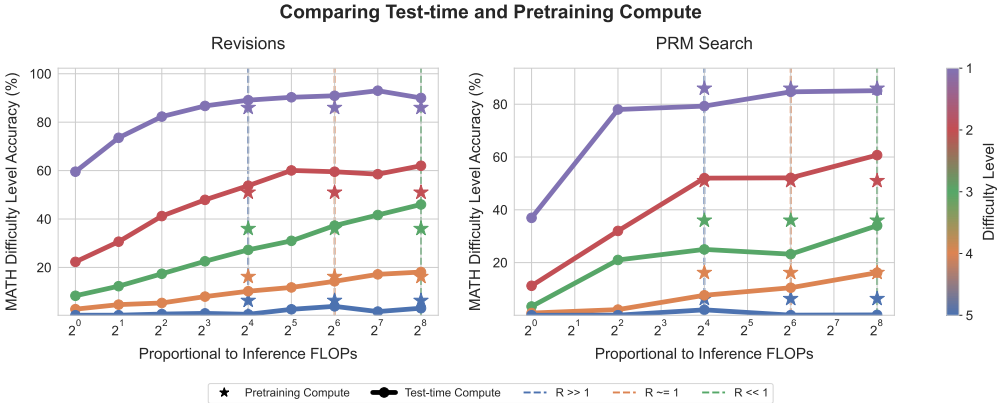
Inference Workload Determines the Exchange Rate

$$R = \frac{D_{\text{inference}}}{D_{\text{pretrain}}}, \qquad a = M + \frac{3(M-1)}{R}$$

- Low R: training savings are spread over fewer inference tokens, allowing more extra compute per request.
- High R: the same savings are spread across a larger inference workload.

Workload	Low	Comparable	High
R	0.16	0.79	22
a, for $M \approx 14$	≈ 258	≈ 63	≈ 16

Small Model with Extra Compute versus Larger Model



Curves Small model with adaptive revisions (left) or PRM search (right).
Stars Greedy accuracy of the $\sim 14\times$ larger model.
Compare the same color at each workload's vertical line. A curve above its star favors additional inference. Colors use oracle difficulty bins.

Limits of the Comparison

- Estimating difficulty needs substantial compute, excluded from reported budgets.
- Hardest problems gain little from the tested search and revision methods.
- The evidence comes from PaLM 2-S* on MATH, with capability-specific fine-tuning.
- The FLOPs comparison scales parameters at fixed training data and uses oracle difficulty bins.

Conclusions

- The best use of inference compute depends on both the question and the budget.
- Difficulty-based allocation improves PRM search and sequential/parallel revisions, with up to fourfold generation-budget savings in reported comparisons.
- Extra inference can substitute for larger models on some problems and workloads. The hardest problems remain a limitation.

The title's claim is conditional on problem difficulty and expected inference workload.

Thank you

Questions and Discussion

Backup: Difficulty Bins and Cross-Validation

1. Score every question using 2,048 samples. Sort and divide into five quantiles.
2. Within a bin, split questions into folds A and B.
3. Use fold A to select settings at each budget. Evaluate those settings on B.
4. Swap A and B. Average held-out performance.

Oracle and predicted difficulty repeat this procedure with different scoring rules.

Backup: Training Rollouts and Lookahead

PRM training Sample multiple continuations from a prefix to estimate expected final correctness. This supplies supervision.

Lookahead search At inference time, generate up to k extra steps with temperature 0. Use the trained PRM's score at that endpoint to evaluate the current prefix.

The temporary lookahead steps help choose which current prefixes to retain. Beam search is the $k = 0$ case.

Backup: Deriving the Inference Multiplier

$$X + aY = M(X + Y)$$

$$a = M + (M - 1)\frac{X}{Y}$$

$$\frac{X}{Y} = \frac{6PD_{\text{pretrain}}}{2PD_{\text{inference}}} = \frac{3}{R}$$

$$\boxed{a = M + \frac{3(M-1)}{R}}$$

Approximate FLOPs per token: forward $2P$, backward $4P$, training total $6P$.